

The Role of AI and LLMs in Developing Parallel Code

1st Dipika Bogati
Computer Science
Bowling Green State University
Bowling Green, USA
dbogati@bgsu.edu

2nd Huy Le Vuong
Computer Science
Bowling Green State University
Bowling Green, USA
hvuong@bgsu.edu

3rd Radhika Srivastava
Computer Science
Bowling Green State University
Bowling Green, USA
rsriva@bgsu.edu

Abstract—

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

II. RELATED WORK

The paper [10] show how AI can speed up the process of finding efficient GPU code. Traditional tools like TVM/Ansor test thousands of different ways to run the same program on a GPU, which can take 15-30 minutes for a single task. This research makes the process faster by introducing a simpler model that focuses on seven key hardware factors, such as data movement and thread utilization together with a hill-climbing style search strategy that steadily moves toward better versions instead of relying on random guesses. With these improvements, the system can generate superior quality code in only 1-2 minutes.

[20] present a survey of how AI techniques can address the long-standing challenges of parallel programming, such as dividing tasks across processors, managing memory, and ensuring correct synchronization. The paper reviews three main AI approaches. Genetic algorithms are described as effective for exploring large and complex solution spaces in areas such as task distribution, synchronization strategies, and memory allocation, although they can suffer from slow convergence and variability across runs. Reinforcement learning is seen as a promising way to dynamically manage resources and threads where agents can learn by trial and error how to balance workloads, create or terminate threads, and adapt to changing system conditions. Neural networks are positioned as especially useful for automatic parallel code generation, and performance prediction.

New techniques like transfer learning and neurosymbolic methods make AI tools even more powerful. Transfer learning helps models adapt to different types of tasks without starting from scratch, while neurosymbolic methods improve accuracy and make results easier to explain, especially in tasks such as debugging and error detection.

The paper also points out several practical ways AI can help in parallel programming. AI can automatically turn sequential programs into parallel programs that run efficiently on specific hardware. It can also manage resources on the fly, deciding how to assign tasks and memory to different cores depending

on the workload which makes the system faster and more efficient.

The paper [12] assess the ability of LLMs to generate parallel code using OpenMP, focusing on chatGPT 3.5 and GitHub Copilot. They tested both tools on nine different applications, ranging from simple numerical tasks to more complex simulations, by asking them to transform sequential programs into parallel versions. The results show that both models can produce working parallel code, but with notable differences in quality. chatGPT was able to generate correct solutions in simpler cases, though it often required multiple rounds of synchronization. GitHub Copilot, on the other hand, provided more reliable output, needed less user intervention, and often achieved performance closer to manually optimized code. However, they are not yet a substitute for human expertise in performance-critical applications.

LLMCompiler is a system proposed by Kim et al. (2024) that allows for parallel function calls instead of sequential, improving the way LLMs interact with external tools. Current methods, like the ReAct process tool, invoke the process one step at a time, increasing latency, cost and the possibility of errors such as incomplete calls. LLM compiler addresses these limitations through a compiler inspired design that includes a planner to map tasks and their dependencies, a dispatcher to manage task execution as inputs become available, and an executor that runs tasks in parallel [9].

To generate the high performance parallel code generation, authors have created a new architecture which has four Parallel Code Generation Agents: Memory, Planning, Tools and Action which are connected to the main Agent [7]. Memory agent unifies the internal knowledge generated from the Fine-Tuning and external knowledge generated from Retrieval-Augmented Generation (RAG). Planning agent includes the prompting task which consists of both single-stage prompt and multi-stage prompt. Tools is used in the verification pipeline where the generated C++ code is compiled using GCC, the abstract Syntax Tree (AST) is employed in nm tool. Action defines the agent's capabilities for state transitions (edits, evaluations, resets) during the process of code optimization. They did the data augmentation on the publicly available **PIE** dataset using open-source model and the APIs to synthesize the code. To actually increase the performance of the parallel

code, planning-oriented prompting (prompt strategies: parallel, serial, any and multi-stage), code verification and Retrieval Augmented Editing are implemented. The experimental results shows that x6.06 speedup is achieved in Qwen2.5-Coder-7B-Instruct and x5.13 speedup is achieved in Qwen2.5-Coder-14B-Instruct LLM models [7].

Research was performed to design LLM on solving the High Performance Computing (HPC) [14]. The HPC code is generated from the GitHub repositories with the rating greater than 3. The HPC Code are filtered using deduplication, and files over 1MB and under 15 tokens are also removed. Using this dataset, three language models: GPT-2 with 1.5B parameters, GPT-Neo with 2.7B parameters, and PolyCoder with 2.7B parameters are fine-tuned. Using the fine-tuned models, set of HPC functions are generated(text generation) and performance and correctness of the generated code are evaluated. In addition, models' ability to label the for loops are also evaluated in OpenMP pragmas for which **for loops** are taken as input and OpenMP pragma are generated as the output while fine-tuning. The experiment results shows that the generated code speedup is always > 1 which means the LLM generated HPC code is correct and parallelized [14].

To improve Parallel Program Performance, high-quality mapper is required which assigns tasks to processors and data to memory properly which requires deep knowledge about hardware, low-level system APIs and applications which is difficult for the domain professionals. So, the paper [19] proposed a new framework to automate the generation and optimization of mapper code using LLMs using Agent-System Interface. One of the core Agent-System Interface is Domain-Specific Language which has declarative design (allowing users to declare what to achieve instead of long C++ code). Then AutoGuide is used to take in the raw logs and error messages from the mapper and translates them into the clear guidance. Finally, generative optimization loop is implemented which takes two inputs: application metadata and server specifications which define the search space explored by agents in the process of optimization. Hence, by utilizing the given inputs, the agent produces the mapper code that is executed on the application code in server. Authors have compared the results generated from their approach with OpenTuner and human expert and concluded that their method achieve 3.8 x faster in 10 iterations compared to 1000 iterations in OpenTuner and 1.34 x higher speedup than human expert written mappers [19].

A study was performed to analyze if Large Language Models (LLMs) could write the efficient and faster parallel code. The benchmarking framework called as PAREVAL [13] evaluates the effectiveness of state-of-the-art closed-source and open-source models where the study shows that GPT-3.5 performs best with serial parallel code generation. The observation from the study also proves that LLMs struggle in performing the code generation for MPI and performs best in the code generation for OpenMP and Kokkos code generation. Also, the studies says that LLMs generate the correct code but not high-performant code as the same time. So, the paper

suggests to provide the serial portion of the code to generate the high-performant parallel code generation from LLMs [13].

The study is done on ChatGPT for converting the serial code to the parallel code. For the code references, authors have taken the python code where ChatGPT is expected to produce the multi-processing code (multithreading and multi-processing). The study has proved that ChatGPT is effective in inserting the parallel code compared to human experts which is good at reducing the manual efforts. However, ChatGPT fails to produce the effective parallel code for the complex algorithm or code base if ChatGPT is not directed by the correct patterns [4]

There were several works on building/fine-tuning a domain-specific language models for OpenMP pragma generation mostly using the HPCorpus dataset. Those domain-specific LLMs demonstrates superior performance over existing language models in HPC contexts (Chen et al. 2024 [3]) and (Schneider et al. 2024 [16])

Chen et al. (2024) introduce OMPGPT, a novel domain-specific language model designed specifically for OpenMP pragma generation in high-performance computing (HPC). While general code-based LLMs like StarCoder and CodeLlama are useful for many programming tasks, HPC has unique requirements that benefit from a smaller, specialized model such as OMPGPT. OMPGPT used 'Chain-of-OMP', an innovative prompt engineering strategy to further enhance the effectiveness of the LLM. OMPGPT outperforms existing large language models specialized in OpenMP tasks. Additionally, OMPGPT is notably small in size, making it more suitable for hardware constraints typical in HPC environments [3].

The use case of the LLM is implemented in assessment of the parallel programming. The research [6] proposed a GreAlter which is auto-grading system for parallel programming assessment. The LLM-based system, GreAlter, is heavily based on prompt engineering techniques. GreAlter is built on ChatGPT-4 which inputs the JSON file with structured grading requirements from the rubrics and outputs the code marked as "correct" or "incorrect" and generates the summary of mistakes done by the students. This helps teachers and teaching assistants to produce unbiased students' final grades with minimal time consumption.

Another important use case in parallel programs is to identify the part of the program that slows down its overall work to implement it in parallel programming. So, the study [1] evaluated LLMs to profile data of the kernel source and found out that SoTA LLMs can understand the roofline problems and can categorize the program's arithmetic intensity. Compared to zero-shot prompting, few-shot prompting has improved in predicting the OMP programs and CUDA's arithmetic intensity. Also, they proved that fine-tuning did not work out that much as the dataset has only 340 data points.

Since existing static tools have limited effectiveness, and general-purpose language models like GPT-3.5 perform poorly on MPI code generation compared to standard programming tasks, Schneider et al. 2024 show that domain-specific models outperform larger, general models. They introduce MPIrigen,

a model fine-tuned on a dataset containing MPI-related C/C++ code (HPCorpus MPI). MPIrigen significantly outperforms GPT-3.5 in generating accurate MPI function calls which proves domain-specific fine-tuned LLMs is more suitable for parallel computing code generation [16].

Kadosh et al., 2024 introduce OMPar, an AI-driven tool that automates the parallelization of C/C++ code using OpenMP pragmas. OMPar leverages Large Language Models (LLMs) through two main components: OMPify, which evaluates the potential for loop parallelization, and MonoCoder-OMP, a fine-tuned model that generates accurate OpenMP pragmas. Results show that OMPar outperforms existing methods in identifying parallelizable loops and generating efficient pragmas. Additionally, OMPar can handle partial or incomplete codebases and continuously improve by learning from new code patterns [8].

Nichols et al., 2024 use LLMs for automating complex development and analysis tasks in HPC. The authors introduce a new dataset of HPC and scientific codes and use it to fine-tune several pre-trained LLMs called HPC-Coder, a model specifically fine-tuned on parallel codes. Experiments show that HPC-Coder can auto-complete HPC functions (where generic models cannot), add OpenMP pragmas to loops, and model performance changes in scientific application repositories and programming competition solutions [15].

Sollenberger et al. (2025) examined how large language models can help automate the process of creating and checking compiler tests for directive-based parallel programming models like OpenMP and OpenACC. Since writing these tests manually takes a lot of time and effort, they proposed using two LLMs together. One to generate the tests and another to evaluate their correctness. The authors tested several open-source models of different sizes and compared their ability to write working code and identify valid or invalid programs. Their experiments showed that the Deepseek-Coder-33B-Instruct model produced the most reliable compiler tests, while Qwen2.5-Coder-32B-Instruct was the best at evaluating them [17].

Building on this idea, Dearing et al. (2025) explored how large language models can also help improve the energy efficiency of parallel scientific codes. They introduced a system called LASSI EE that uses LLMs to automatically refactor programs so that they run more efficiently without changing their results. The authors tested the framework on 20 CUDA C++ benchmark programs from the HeCBench Suite, running on NVIDIA A100 GPUs. LASSI EE reduced energy use in 85 percent of the tested codes, with an average improvement of 47 percent in energy efficiency. It includes useful features such as power profiling, automated debugging, and an LLM as a judge component that checks the correctness of refactored code. The framework also identifies optimization strategies like better memory management, and asynchronous operations [5].

TehraniJamsaz et al. (2024) developed a system called CodeRosetta that uses large language models to translate code between general programming languages and high performance computing versions. It mainly focuses on converting

between C++, CUDA, and Fortran, which helps older scientific programs run more efficiently on modern GPUs. Instead of relying on manually labeled examples or predefined translation rules, CodeRosetta learns patterns in the code on its own by analyzing syntax and structure. It also improves its translations through noise injections and back translation, allowing it to refine the output automatically. The system outperformed earlier approaches such as BabelTower and TransCoder, achieving much higher accuracy and producing CUDA code that compiled successfully in nearly all cases [18].

Mahmud et al. (2025) introduce AutoParLLM, a novel framework that combines Graph Neural Networks (GNNs) with Large Language Models (LLMs) to automate the parallelisation of sequential code, specifically targeting OpenMP directives. AutoParLLM utilises GNNs to generate context for the LLMs, which significantly enhances their ability to produce efficient parallel code, outperforming state-of-the-art LLMs like GPT-4 by substantial margins on the NAS Parallel Benchmark and Rodinia Benchmark suites in terms of both code quality metrics and execution speedup. Furthermore, the researchers propose a new evaluation metric called OMP-SCORE to accurately assess the quality of generated OpenMP constructs. The work also demonstrates the framework's extensibility to other parallel programming models, such as OpenACC [11].

Le Chen et al. (2025) introduces PCEBENCH, a novel, multi-dimensional benchmark designed to evaluate Large Language Models (LLMs) specifically on their ability to generate efficient and correct parallel code, focusing on OpenMP and Message Passing Interface (MPI) paradigms crucial for High-Performance Computing (HPC). The paper outlines the benchmark's comprehensive metrics, which assess aspects such as compilability, executability, data race freedom, functional correctness, and speedup over sequential implementations. Utilising an LLM agent-based pipeline for data generation and verification, the research compares several state-of-the-art LLMs, finding that models like GPT-4o generally outperform others, although all models face significant challenges, particularly with MPI code generation. Furthermore, the study explores the importance of prompt engineering and LLM capability in fixing errors within parallel code [2].

III. PROPOSAL

Accurately assessing the performance of parallel GPU code typically requires execution-time profiling on target hardware which is a process that is often impractical due to limited access to high-end GPUs. In this paper, we will investigate whether Large Language Models (LLMs) can provide an alternative approach to GPU performance prediction without direct hardware execution. With a given the source code of a GPU kernel and the specifications of a target GPU, we will try to prompt the LLM to determine whether the kernel is compute-bound or memory-bandwidth-bound.

To explore this question, we found a dataset of 340 GPU kernels written in CUDA and OpenMP, each labeled using empirical profiling results. We will evaluate LLMs under four

scenarios: (1) with access to profiling data, (2) zero-shot with source code only, (3) few-shot with example code-label pairs, and potentially (4) fine-tuned on a small custom dataset.

Parallelized code often places significant demands on a GPU’s memory bandwidth and computational capacity (FLOP rate). We will classify such code as either memory-bandwidth-intensive or computation-intensive based on its arithmetic intensity (AI)—the number of operations performed per byte of memory traffic. Code with low AI is considered memory-bandwidth-intensive, while code with high AI is considered computation-intensive.

We plan to use features derived from the code—arithmetic intensity (FLOPs per byte) and achieved performance (GFLOP/s)—together with hardware specifications—memory bandwidth (GB/s) and compute capability (GFLOP/s)—LLMs can potentially predict whether a given parallelized kernel is limited by memory bandwidth or compute power. This capability can help users identify the most suitable GPU for their workloads based on the desired performance characteristics.

REFERENCES

- [1] Gregory Bolet, Giorgis Georgakoudis, Harshitha Menon, Konstantinos Parasyris, Niranjan Hasabnis, Hayden Estes, Kirk Cameron, and Gal Oren. Can large language models predict parallel code performance? In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, pages 1–6, 2025.
- [2] Le Chen, Nesreen Ahmed, Mihai Capota, Ted Willke, Niranjan Hasabnis, and Ali Jannesari. PCEBench: A Multi-Dimensional Benchmark for Evaluating Large Language Models in Parallel Code Generation. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 546–557, Los Alamitos, CA, USA, June 2025. IEEE Computer Society.
- [3] Le Chen, Arijit Bhattacharjee, Nesreen K. Ahmed, Niranjan Hasabnis, Gal Oren, Vy A. Vo, and Ali Jannesari. Ompgpt: A generative pre-trained transformer model for openmp. In *European Conference on Parallel Processing*, 2024.
- [4] Denny B. Czejdo, Wiktor B. Daszczuk, and Waldemar Grabski. Using chatgpt for converting sequential python programs into parallel code. In Wojciech Zamojski, Jacek Mazurkiewicz, Jaroslaw Sugier, Tomasz Walkowiak, and Janusz Kacprzyk, editors, *Advances in Dependable Systems and Networks*, pages 41–50, Cham, 2025. Springer Nature Switzerland.
- [5] Matthew T. Dearing, Yiheng Tao, Xingfu Wu, Zhiling Lan, and Valerie Taylor. Leveraging llms to automate energy-aware refactoring of parallel scientific codes, 2025.
- [6] Skyler Grandel, Douglas C Schmidt, and Kevin Leach. Applying large language models to enhance the assessment of parallel functional programming assignments. In *Proceedings of the 1st International Workshop on Large Language Models for Code*, pages 102–110, 2024.
- [7] Xiaowen Huang, Xu Zhang, Lvfang Tao, Renjie Mao, Nan Zhou, Wenxi Zhu, Minwen Deng, Jintao Meng, Yanjie Wei, Amelie Chi Zhou, et al. Paracoder: Parallel code generation with large language model. In *Proceedings of the 1st FastCode Programming Challenge*, pages 1–7, 2025.
- [8] Tal Kadosh, Niranjan Hasabnis, Prema Soundararajan, Vy A. Vo, Mihai Capota, Nesreen K. Ahmed, Yuval Pinter, and Gal Oren. Ompar: Automatic parallelization with ai-driven source-to-source compilation. *ArXiv*, abs/2409.14771, 2024.
- [9] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling, 2024.
- [10] Chendi Li, Yufan Xu, Sina Mahdipour Saravani, and Ponnuswamy Sadayappan. Accelerated auto-tuning of gpu kernels for tensor computations. In *Proceedings of the 38th ACM International Conference on Supercomputing*, ICS ’24, page 549–561, New York, NY, USA, 2024. Association for Computing Machinery.
- [11] Quazi Ishtiaque Mahmud, Ali TehraniJamsaz, Hung Phan, Le Chen, Mihai Capotă, Theodore Willke, Nesreen K. Ahmed, and Ali Jannesari. Autoparllm: Gnn-guided context generation for zero-shot code parallelization using llms, 2025.
- [12] Marko Mišić and Matija Dodović. An assessment of large language models for openmp-based code parallelization: a user perspective. *Journal of Big Data*, 11, 11 2024.
- [13] Daniel Nichols, Joshua H Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. Can large language models write parallel code? In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, pages 281–294, 2024.
- [14] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. Modeling parallel programs using large language models. *arXiv preprint arXiv:2306.17281*, 2023.
- [15] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Gamblin, and Abhinav Bhatele. Hpc-coder: Modeling parallel programs using large language models. In *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*, pages 1–12, 2024.
- [16] Nadav Schneider, Niranjan Hasabnis, Vy A. Vo, Tal Kadosh, Neva Krien, Mihai Capotua, Abdul Wasay, Guy Tamir, Ted Willke, Nesreen K. Ahmed, Yuval Pinter, Timothy Mattson, and Gal Oren. Mpirigen: Mpi code generation through domain-specific language models. *Proceedings of the 2024 Workshop on AI For Systems*, 2024.
- [17] Zachariah Sollenberger, Rahul Patel, Saieda Ali Zada, and Sunita Chandrasekaran. Llm4vv: Evaluating cutting-edge llms for generation and evaluation of directive-based parallel programming model compiler tests, 2025.
- [18] Ali TehraniJamsaz, Arijit Bhattacharjee, Le Chen, Nesreen K. Ahmed, Amir Yazdanbakhsh, and Ali Jannesari. Coderosetta: Pushing the boundaries of unsupervised code translation for parallel programming, 2024.
- [19] Anjiang Wei, Allen Nie, Thiago SFX Teixeira, Rohan Yadav, Wonchan Lee, Ke Wang, and Alex Aiken. Improving parallel program performance with llm optimizers via agent-system interfaces. *arXiv preprint arXiv:2410.15625*, 2024.
- [20] Rimma Zaripova, Adam Mentsiev, and Olga Kovrizhnykh. Advancing parallel programming integrating artificial intelligence for enhanced efficiency and automation. *E3S Web of Conferences*, 460:1–9, 12 2023.